Bellman-Guided Warm-Start Configuration of LSTM Networks for ECG Forecasting

Anonymous Author(s)
Affiliation withheld for blind review
Email: author@institution.edu

***Abstract*—Configuring a recurrent forecaster is costly because each candidate architecture ordinarily requires an independent training run. This paper investigates a compact alternative in which tabular Q-learning searches an LSTM design space while compatible candidates inherit the preceding checkpoint. Hidden width, recurrent depth, and dropout define an 18-state Markov decision process, and seven local actions modify one coordinate at a time. After each action, the selected model is trained for three epochs, evaluated on 5,000 validation windows, and assigned the reward $r = -\mathrm{MSE}$. Shape-compatible transitions reuse learned parameters; all other transitions restart from a new initialization. The recorded ECG5000 experiment contains 5,000 traces, 600,000 length-20 forecasting windows, and 12 search episodes on a CPU. The best Adam-based warm-start checkpoint achieves an MSE of 0.088472 and $R^2 = 0.9133$. Relative to the notebook's cold-start, manual-SGD search, MSE decreases from 0.943495 by a factor of 10.66, whereas MAE decreases from 0.727927 to 0.195127 (3.73$\times$) at the same checkpoint. Thus, the observed tenfold gain applies to MSE, not MAE. Because the two runs also differ in optimizer, initialization policy, and effective training budget, the improvement is attributed to the complete warm-start search procedure rather than to Q-learning alone. The study demonstrates the promise of inexpensive sequential refinement while identifying the controls required for a definitive evaluation.**

***Index Terms*—electrocardiogram, LSTM, Q-learning, Bellman equation, hyperparameter optimization, time-series forecasting, weight reuse**

I. Introduction

Electrocardiograms (ECGs) combine sharp local morphology with longer temporal structure, making them a useful test bed for sequential models. Public repositories such as PhysioNet [1] and the UCR Time Series Archive [2] have made reproducible ECG experiments widely accessible. Although ECG5000 is commonly treated as a classification benchmark, each trace can also define a self-supervised forecasting task: given the previous samples, predict the next standardized amplitude. This formulation evaluates short-horizon temporal modeling without using the class label as supervision.

Long short-term memory (LSTM) networks [3] are well suited to this setting, but their performance depends strongly on architectural and training choices. Hidden width determines representational capacity, depth changes the recurrent hierarchy, and dropout controls inter-layer regularization. Optimizer and training budget further affect the quality of a candidate. Prior empirical work confirms that seemingly modest LSTM design choices can produce substantial performance differences [4]. Exhaustively retraining every configuration is therefore both expensive and wasteful.

Modern hyperparameter optimization (HPO) offers several alternatives. Random search provides a competitive baseline [5]; Bayesian optimization models validation performance with a surrogate [6]; and successive-halving methods concentrate resources on promising candidates [7], [8]. BOHB combines model-based selection with adaptive resource allocation [9]. Reinforcement learning (RL) provides a complementary formulation in which configuration decisions form a sequence and validation quality supplies the reward, an idea used extensively in neural architecture search (NAS) [10].

Most HPO methods evaluate configurations as independent points or require a substantially larger search budget. This paper instead examines a small, interpretable design space in which a Q-learning agent makes local changes and the forecaster reuses weights whenever consecutive architectures are structurally compatible. The study is grounded in the supplied notebook and makes three contributions:

- an interpretable 18-state Markov model of LSTM width, depth, and dropout with seven local controller actions;
- a shape-aware warm-start rule that preserves compatible parameters across sequential evaluations; and
- a notebook-grounded analysis that distinguishes the 10.66$\times$ MSE reduction from the 3.73$\times$ MAE reduction at the MSE-selected checkpoint.

The experiment is intended as a reproducible proof of concept, not a clinical assessment. Both comparison runs already use Q-learning, while differing in optimizer, checkpoint reuse, and cumulative training. Accordingly, the reported improvement characterizes the combined warm-start pipeline and is not presented as the isolated effect of RL.

II. Related Work

A. Deep Models for Time Series

LSTM introduced gated memory to preserve useful gradients over long sequences [3]. Subsequent comparisons of LSTM variants showed that careful tuning often matters more than elaborate cell modifications [4]. Hybrid models such as LSTM–FCN combine recurrent and convolutional pathways for time-series classification [11], while broad evaluations across UCR datasets document both the strength and sensitivity of deep architectures [12]. Forecasting surveys reach a similar conclusion: recurrent networks remain competitive, but reliable results depend on disciplined preprocessing, validation, and model selection [13], [14].

The model studied here is intentionally simpler: a configurable LSTM feeds a scalar linear head that predicts the next ECG sample. Dropout is included because it can reduce co-adaptation [15]; in the underlying implementation, however, recurrent dropout is active only when the LSTM contains more than one layer.

B. Sequential Configuration Search

Bellman's formulation of sequential decision making [16] provides a natural basis for configuration search. Q-learning estimates action values without an explicit model of the environment [17]. At a much larger scale, RL-based NAS uses learned controllers to generate architectures [10]; ENAS lowers this cost through parameter sharing [18]; and network-transformation methods preserve useful weights while altering a model graph [19]. The present method adopts the same principle of computational reuse, but restricts it to consecutive LSTM configurations with identical parameter shapes. Adam [20] performs the short warm-start updates.

This search lies between black-box HPO and full NAS. Each action modifies at most one coordinate within a fixed LSTM family, making the transition interpretable and the value function small enough to store in a table. The tradeoff is expressiveness: the controller cannot invent new recurrent cells, feature extractors, or skip connections. The objective is therefore not to replace general HPO, but to determine whether Bellman-guided local search and conservative checkpoint reuse can improve a tightly constrained forecasting workflow.

III. Methodology

A. Problem Statement

Let $\mathcal{D}_{\rm tr}$ and $\mathcal{D}_{\rm val}$ denote the training and validation sets of input windows and next-sample targets. For an LSTM configuration $\lambda = (H, L, D)$, training procedure $\mathcal{A}$, and parameters θ, the validation objective is

$$\mathcal{L}_{\rm val}(\lambda,\theta) = \frac{1}{|\mathcal{D}_{\rm val}|} \sum_{(\mathbf{X},y)\in\mathcal{D}_{\rm val}} \left(y - f_{\lambda,\theta}(\mathbf{X})\right)^2. \tag{1}$$

Conventional HPO seeks $\lambda^* = \arg\min_{\lambda\in\Lambda} \mathcal{L}_{\rm val}(\lambda, \mathcal{A}(\lambda))$ and evaluates each λ independently. Here, the next candidate may inherit the immediately preceding checkpoint. Its performance therefore depends on both the selected configuration and the path used to reach it. The controller optimizes the finite-horizon return

$$G_k = \sum_{j=0}^{K-k-1} \gamma^j r_{k+j+1}, \qquad r_{k+1} = -\mathcal{L}_{{\rm val},k+1}, \tag{2}$$

over $K = 12$ episodes. Model selection is based on the minimum recorded validation MSE, not the terminal state, because a late exploratory move may be worse than an earlier checkpoint.

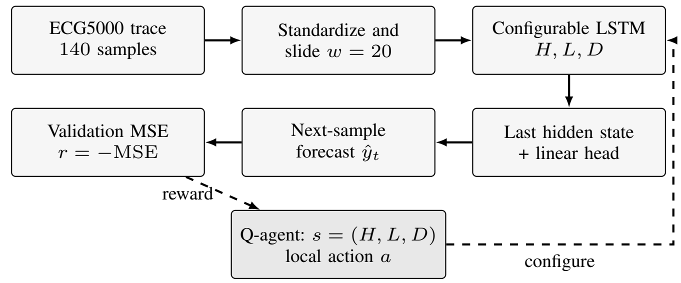

Fig. 1. Block-wise architecture of the Q-guided ECG forecaster. Solid arrows form the prediction path; dashed arrows close the search loop.

B. Forecasting Task and LSTM

Let $\mathbf{x} = (x_1, \ldots, x_{140})$ be a standardized ECG trace. With window length $w = 20$, each valid position yields the supervised pair

$$\mathbf{X}_t = [x_{t-w}, \ldots, x_{t-1}], \qquad y_t = x_t. \tag{3}$$

The LSTM consumes $\mathbf{X}_t$ sequentially. Its gated update can be written compactly as

$$\begin{aligned}
\mathbf{i}_t &= \sigma(W_i x_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i),\\
\mathbf{f}_t &= \sigma(W_f x_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f),\\
\mathbf{o}_t &= \sigma(W_o x_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),\\
\tilde{\mathbf{c}}_t &= \tanh(W_c x_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c),\\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),
\end{aligned} \tag{4}$$

The final hidden state is mapped to $\hat{y}_t = \mathbf{w}^\top \mathbf{h}_t + b$ by a linear regression head. Candidate quality is measured by

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y}_n)^2, \quad \mathrm{MAE} = \frac{1}{N}\sum_{n=1}^{N}|y_n - \hat{y}_n|. \tag{5}$$

C. LSTM Configuration as an MDP

The controller state $s = (H, L, D)$ records the hidden width, number of recurrent layers, and dropout rate:

$$H \in \{32, 64, 128\}, \quad L \in \{1, 2\}, \quad D \in \{0, 0.2, 0.4\}. \tag{6}$$

These choices define 18 states. The seven-action set increases or decreases one coordinate, or leaves the configuration unchanged. Values are clipped at the search-space boundaries, so an outward action may become a self-transition. Figure 1 shows how the deterministic transition $T(s, a)$ controls the forecasting model and receives validation feedback.

After the LSTM specified by $s' = T(s, a)$ is trained, the environment returns $r = -\mathrm{MSE}_{\rm val}$. The agent then applies the one-step Q-learning update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right], \tag{7}$$

with learning rate $\alpha = 0.4$ and discount factor $\gamma = 0.8$. An ϵ-greedy policy uses $\epsilon = 0.5$ to alternate between random exploration and the highest-valued action. The table is initialized to zero, the search begins at $(64, 1, 0)$, and 12 episodes are executed.

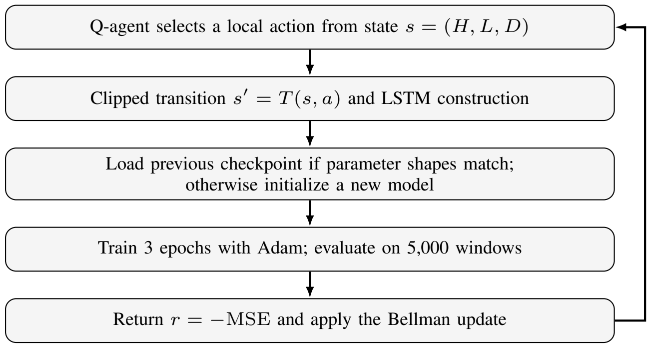

Fig. 2. One episode of the warm-start Q-learning search.

Algorithm 1 Warm-Start Q-Learning for LSTM Configuration

Require: $\mathcal{D}_{\mathrm{tr}}$, $\mathcal{D}_{\mathrm{val}}$, state set $\mathcal{S}$, actions $\mathcal{U}$
Ensure: Best checkpoint b^* and episode log $\mathcal{B}$
Set $s_0 = (64, 1, 0)$, $Q(s, a) = 0$, $\theta^- \leftarrow \varnothing$, $\mathcal{B} \leftarrow \varnothing$
for $k = 1, \ldots, 12$ **do**
  Select a_k by an ϵ-greedy policy, $\epsilon = 0.5$
  Set $s_{k+1} \leftarrow T(s_k, a_k)$ and construct its LSTM
  Load θ^- if parameter shapes match; otherwise reinitialize
  Train three epochs with Adam on $\mathcal{D}_{\mathrm{tr}}$
  Evaluate MSE, RMSE, MAE, and R^2 on $\mathcal{D}_{\mathrm{val}}$
  Set $r_k \leftarrow -\mathrm{MSE}_k$ and update Q using (7)
  Save the checkpoint; append (s_{k+1}, a_k, r_k) and metrics to $\mathcal{B}$
end for
return $b^* \leftarrow \arg\min_{b \in \mathcal{B}} \mathrm{MSE}(b)$

D. Compatible-Weight Reuse

Figure 2 summarizes a search episode. The candidate is first constructed from s'. If every state-dictionary tensor matches the preceding checkpoint in shape, the stored parameters are loaded before three additional epochs; otherwise, the model is reinitialized. Consequently, repeated visits to $(128, 1, 0)$ during episodes 3–6 continue the same optimization trajectory instead of discarding prior work. Changing the nominal dropout of a one-layer LSTM is also shape-compatible, although recurrent dropout is inactive for $L = 1$. Unlike ENAS-style supernet sharing [18], this mechanism transfers weights only between consecutive candidates.

IV. Experimental Protocol

The notebook downloads the ECG5000 training and test files from the UCR archive, concatenates all 5,000 traces, and removes the class labels from the 140-sample signals. It then standardizes each temporal position over the complete matrix. Applying (3) produces 120 examples per trace and 600,000 windows in total. A seeded 80/20 random split is performed at the window level; to keep the CPU experiment tractable, only the first 20,000 training and 5,000 validation windows are used. The remaining settings are summarized in Table I.

Both recorded searches use the same state space, action set, and reward. The baseline reinitializes every candidate and performs three epochs with a manually implemented SGD update; the proposed variant combines Adam with compatible checkpoint reuse. MSE, root MSE (RMSE), MAE, and R^2 are computed on the validation subset. The notebook does not report elapsed time, the random seeds used for initialization or action selection, repeated trials, confidence intervals, or an untouched final test result. No unrecorded quantities are inferred.

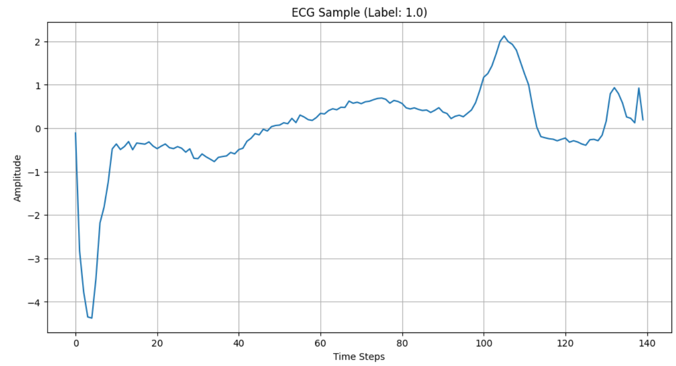

Fig. 3. An ECG5000 trace visualized in the source notebook. The class label is not used by the forecasting target.

TABLE I
Recorded Notebook Configuration

Setting	Value
Dataset / traces / length	ECG5000 / 5,000 / 140
Forecast window / target	20 / next sample
Generated / used windows	600,000 / 20,000 train, 5,000 val.
Batch size / device	512 / CPU
Episodes / epochs per episode	12 / 3
Agent $(\alpha, \gamma, \epsilon)$	$(0.4, 0.8, 0.5)$
Warm-start optimizer / rate	Adam / 10^{-3}
Baseline optimizer / rate	manual SGD / 10^{-3}
Selection objective	minimum validation MSE

In addition to (5), the recorded metrics are

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad R^2 = 1 - \frac{\sum_n (y_n - \hat{y}_n)^2}{\sum_n (y_n - \bar{y})^2}. \tag{8}$$

MSE serves as both the training-aligned selection objective and the controller reward. MAE provides a complementary linear-error measure, while RMSE restores the scale of the standardized target. Because RMSE is the square root of MSE, a $10.66\times$ MSE reduction necessarily corresponds to a $\sqrt{10.66} = 3.27\times$ RMSE reduction rather than another tenfold change.

A. Model Size and Evaluation Budget

For scalar input and output, a one-layer LSTM contains $4H^2 + 13H + 1$ trainable parameters. A two-layer model has $12H^2 + 21H + 1$, including both recurrent layers and the linear head. As Table II shows, the design space ranges from 4,513 to 199,297 parameters. Width and depth therefore change the memory and computational requirements substantially; dropout changes neither the parameter count nor the tensor shapes.

With 20,000 training windows and a batch size of 512, each epoch contains 40 mini-batches, including one partial batch. An episode therefore performs 120 gradient updates, and the full search performs 1,440 updates across all instantiated models. When recurrent matrix multiplication dominates, the cost of a

TABLE II
TRAINABLE PARAMETERS BY WIDTH AND DEPTH

H	$L = 1$	$L = 2$	Two-/one-layer ratio
32	4,513	12,961	2.87×
64	17,217	50,497	2.93×
128	67,201	199,297	2.97×

TABLE III
SELECTED WARM-START SEARCH EPISODES

Ep.	(H, L, D)	MSE	RMSE	MAE	R^2	Reuse
1	$(32, 1, 0)$	.2031	.4506	.2720	.8011	No
3	$(128, 1, 0)$	.1350	.3674	.2471	.8678	No
4	$(128, 1, 0)$	.0996	.3156	.2021	.9024	Yes
6	$(128, 1, 0)$	**.0885**	**.2974**	.1951	**.9133**	Yes
8	$(64, 2, 0)$	.1281	.3579	.2267	.8746	No
12	$(64, 1, .2)$	.0910	.3017	**.1932**	.9109	Yes

length-w window is approximately $O(wLH^2)$. A larger study should consequently report elapsed time, energy, parameter count, and update budget alongside forecast error. The present reward optimizes accuracy alone and does not prefer a smaller model when two candidates have equal loss.

V. Results and Discussion

A. Search Trajectory

The trajectory in Fig. 4 exhibits three distinct phases. The first episode selects $(32, 1, 0)$ and reaches an MSE of 0.203070. The controller then moves through 64 to 128 hidden units. Once $(128, 1, 0)$ persists, compatible reuse turns repeated evaluations into continued optimization: from episodes 3 to 6, MSE decreases from 0.134992 to 0.088472 and R^2 increases from 0.8678 to 0.9133. Architecture changes in episodes 7 and 8 force reinitialization and temporarily degrade every error metric. Table III lists the representative checkpoints.

The selected model is not the largest member of the search space. Its 67,201 parameters constitute 33.7% of the 199,297-parameter maximum. By comparison, the two-layer, 64-unit model evaluated at episode 8 contains 50,497 parameters but yields a higher MSE of 0.128084. This observation suggests that the recorded trajectory did not reward capacity alone; however, unequal initialization histories prevent it from serving as a controlled width-versus-depth comparison.

B. Magnitude and Meaning of the Improvement

Table IV compares the checkpoints selected by minimum MSE. The cold-start/manual-SGD run selects $(128, 1, 0.2)$, although dropout is inactive because the model has only one recurrent layer. The warm-start/Adam run selects $(128, 1, 0)$ at episode 6. MSE decreases from 0.943495 to 0.088472, a reduction of 90.62% or 10.66×. At the same checkpoint, RMSE decreases by 69.38% (3.27×) and MAE by 73.19% (3.73×). The minimum MAE appears at episode 12 and equals 0.193173, corresponding to a 3.77× reduction from the baseline value.

Two qualifications are essential. First, the tenfold factor describes MSE; the corresponding MAE factor is 3.73 at

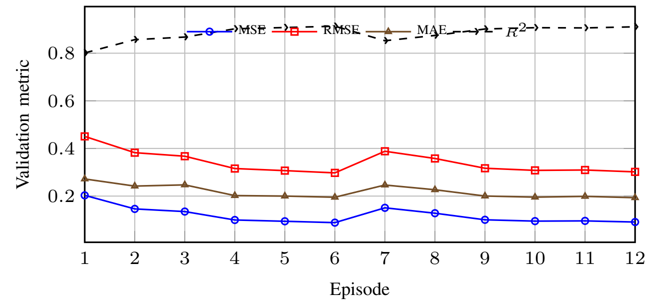

Fig. 4. Recorded validation metrics across the 12 warm-start episodes.

TABLE IV
MSE-SELECTED CHECKPOINT COMPARISON

Search variant	MSE	RMSE	MAE	R^2
Cold start + manual SGD	.943495	.971337	.727927	.0759
Warm start + Adam	**.088472**	**.297443**	**.195127**	**.9133**
Reduction	90.62%	69.38%	73.19%	+0.8374
Factor	10.66×	3.27×	3.73×	—

the MSE-selected checkpoint. Second, the comparison does not isolate Q-learning, because both searches use the same controller. The proposed run simultaneously replaces manual SGD with Adam and allows compatible revisits to continue training. The $(128, 1, 0)$ model, for example, accumulates 12 epochs across episodes 3–6, whereas each cold-start candidate receives only three. The measured gain must therefore be attributed to the combined warm-start/Adam procedure.

The increase in R^2 from 0.0759 to 0.9133 provides a complementary view: on the recorded validation windows, the warm-start predictions follow the observed variance far more closely than the cold-start checkpoint. Nevertheless, these windows are standardized and may overlap within the same trace. The score is therefore a forecasting result on this split, not evidence of patient-level generalization or diagnostic accuracy.

C. Practical Interpretation

The experiment highlights both the benefit and the cost of sequential reuse. A local action space can reach a productive region without exhaustively evaluating all 18 configurations, and a self-transition need not waste the candidate's learned representation. Reuse, however, makes the reward history-dependent: the same state can obtain different losses after different amounts of cumulative training. A more rigorous controller should therefore include checkpoint age or consumed compute in the state and compare candidates under a common budget.

Zero-initialized Q-values introduce an additional exploration bias because all observed rewards are negative. After an action acquires a negative value, an unvisited action at zero appears preferable to the greedy policy even without supporting evidence. This behavior may promote coverage, but 12 episodes cannot visit the 18×7 state–action space or support a convergence claim. Future implementations should randomize

TABLE V
VALIDITY RISKS AND REQUIRED REMEDIES

Risk	Likely effect	Required remedy
Window-level split	correlated train/validation contexts	split by trace before scaling/windowing
Full-data scaling	validation information in normalization	fit scaler on training traces only
One stochastic run	unknown variance and selection stability	multiple seeds and confidence intervals
Unequal warm-start age	confounds architecture and training budget	compare equal updates or add age to state
Optimizer changed	gain cannot be assigned to reuse or RL	factorial Adam/SGD and reuse ablation
Validation-only report	optimistic post-selection estimate	untouched final test and external cohort

ties, decay ϵ, log visitation counts, and reserve separate data for final model selection.

Checkpoint compatibility is currently defined only by tensor shape. Exact loading is valid when state dictionaries match, but structural compatibility does not always imply semantic equivalence. For instance, changing dropout in a one-layer LSTM leaves the tensors unchanged because recurrent dropout is inactive, whereas changing depth invalidates the checkpoint. A richer transfer operator could preserve shared submodules, initialize only the added components, and record the inherited parameter fraction, following ideas from network-transformation search [19].

VI. Validity Threats and Future Work

The principal validity risks arise from data partitioning and experimental control. Standardization is fitted before splitting, and overlapping windows from the same trace are randomly assigned to both pools. Similar temporal contexts may therefore appear in training and validation, producing an optimistic estimate of generalization. A corrected protocol should partition complete traces first, fit the scaler on training traces only, preserve the official test set, and evaluate the selected model once on untouched data.

The recorded evidence also consists of a single stochastic trajectory over truncated subsets. Reliable comparisons require multiple seeds, paired trace-level splits, and confidence intervals. In addition, the two runs jointly change optimizer, initialization policy, and effective epoch budget. A factorial study should separate these effects and include equal-budget random search, Hyperband, BOHB, and a fixed-configuration LSTM. Table V summarizes the minimum controls needed for such an evaluation.

Finally, next-sample error on standardized ECG5000 signals is not a clinical endpoint. Any biomedical claim would require patient-level data, clinically meaningful morphology measures, external cohorts, and a documented inference cost. Reporting both the best search observation and the post-selection test result would distinguish optimization progress from generalization.

A. Reproducibility and Ethical Scope

The notebook provides the data URLs, split seed, subset sizes, model code, controller constants, and per-episode outputs. A complete artifact should also pin software versions, set all Python/NumPy/PyTorch seeds, retain the sampled actions and checkpoints, identify the hardware, and export a machine-readable results table. These additions would enable exact trajectory replay and a fair comparison of computational cost.

The system makes no diagnosis, treatment recommendation, or patient-specific decision. ECG5000 class labels are excluded, and the target is only the next standardized amplitude. Low forecasting error must not be interpreted as clinical sensitivity, specificity, calibration, or safety. The method should remain a signal-modeling experiment unless it is validated against clinically defined endpoints under appropriate governance and domain review.

VII. Conclusion

This paper presented a Bellman-guided procedure for configuring LSTM forecasters while reusing compatible checkpoints. In the 12-episode ECG5000 run, the warm-start/Adam pipeline achieved an MSE of 0.088472 and $R^2 = 0.9133$. Relative to the recorded cold-start/manual-SGD search, MSE improved by 10.66× and MAE by 3.73× at the same checkpoint. The result shows that sequential reuse can make a small search budget productive, but it does not establish that RL alone caused the gain. Trace-level partitioning, repeated trials, equal-budget ablations, and an untouched final test set remain necessary before the improvement can be considered generalizable.

References

[1] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[2] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.

[5] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[6] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 2951–2959. [Online]. Available: https://proceedings.neurips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html

[7] K. Jamieson and A. Talwalkar, "Non-stochastic best arm identification and hyperparameter optimization," in *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 51, 2016, pp. 240–248. [Online]. Available: https://proceedings.mlr.press/v51/jamieson16.html

[8] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. [Online]. Available: https://jmlr.org/papers/v18/16-558.html

[9] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1437–1446. [Online]. Available: https://proceedings.mlr.press/v80/falkner18a.html

[10] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proceedings of the 5th International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1611.01578

[11] F. Karim, S. Majumdar, H. Darabi, and S. Harford, "LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 6, pp. 1662–1669, 2018.

[12] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: A review," *Data Mining and Knowledge Discovery*, vol. 33, no. 4, pp. 917–963, 2019.

[13] H. Hewamalage, C. Bergmeir, and K. Bandara, "Recurrent neural networks for time series forecasting: Current status and future directions," *International Journal of Forecasting*, vol. 37, no. 1, pp. 388–427, 2021.

[14] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20200209, 2021.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[16] R. Bellman, "A markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[17] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[18] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4095–4104. [Online]. Available: https://proceedings.mlr.press/v80/pham18a.html

[19] H. Cai, T. Chen, W. Zhang, Y. Yu, and J. Wang, "Efficient architecture search by network transformation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2787–2794.

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980